

I think this is the perfect last step before coding. From now on, whenever we create a module, we'll refer back to this overview.

1. Market Session Manager

Overview

Knows whether the market should be running or not.

Responsibility

- Detect market open
- Detect market close
- Check trading holidays

Input

- Current date & time
- Trading calendar

Output

- MARKET_OPEN
- MARKET_CLOSE

Problem it solves

Prevents every module from checking the market time separately.

2. Session Manager

Overview

Manages all active user trading sessions.

Responsibility

- Create User Sessions
- Destroy User Sessions

Input

- Market events
- Enabled strategies from the database

Output

- Running User Sessions

Problem it solves

Provides one central place to start and stop all users' trading sessions.

3. User Session

Overview

Represents one user's trading workspace.

Responsibility

- Own the user's Strategy Runtime
- Hold user-specific configuration

Input

- User details
- Strategy configuration

Output

- Running Strategy Runtime

Problem it solves

Keeps every user's trading isolated from others.

4. Subscription Registry

Overview

Tracks which symbols are needed.

Responsibility

- Count symbol subscribers
- Request subscribe/unsubscribe

Input

- Symbol registration/removal

Output

- Subscribe/Unsubscribe requests

Problem it solves

Prevents multiple WebSocket subscriptions for the same symbol.

5. WebSocket Manager

Overview

Communicates with the broker.

Responsibility

- Connect
- Disconnect
- Subscribe
- Receive live ticks

Input

- Broker connection
- Subscription requests

Output

- Tick-by-Tick (TBT) data

Problem it solves

Keeps only one broker connection for the whole application.

6. Tick Cache

Overview

Stores the latest market tick.

Responsibility

- Save latest tick for each symbol

Input

- Live ticks

Output

- Latest tick

Problem it solves

Provides one shared source of live prices.

7. Candle Builder

Overview

Converts ticks into candles.

Responsibility

- Build candles for required timeframes

Input

- Tick stream

Output

- Completed candles

Problem it solves

Builds candles once and shares them with everyone.

8. Candle Cache

Overview

Stores completed candles.

Responsibility

- Keep latest candles in memory

Input

- Completed candles

Output

- Shared candle data

Problem it solves

Strategies don't need to rebuild candles.

9. Indicator Engine

Overview

Calculates shared indicators.

Responsibility

- Calculate EMA10 (Version 1)

Input

- Completed 5-minute candles

Output

- Updated indicator values

Problem it solves

Calculates EMA once instead of once per user.

10. Indicator Cache

Overview

Stores indicator values.

Responsibility

- Save latest EMA10

Input

- Indicator Engine

Output

- Shared EMA values

Problem it solves

All strategies use the same calculated indicator.

11. Symbol Router

Overview

Routes market events.

Responsibility

- Send events to subscribed Strategy Runtimes

Input

- Tick updates
- Candle updates
- Indicator updates

Output

- Events delivered to runtimes

Problem it solves

Only interested strategies receive market updates.

12. Strategy Runtime

Overview

A virtual trader for one user and one strategy.

Responsibility

- Receive market events
- Maintain local state
- Execute strategy logic

Input

- Tick
- Candle
- EMA10
- User configuration

Output

- BUY / SELL / WAIT

Problem it solves

Keeps each user's strategy execution isolated and independent.

13. Trading Engine (Paper Trade - V1)

Overview

Executes the strategy's decision.

Responsibility

- Simulate trade execution

Input

- BUY / SELL signal

Output

- Paper trade

Problem it solves

Allows you to verify the complete trading flow before integrating a real broker.

14. Trading Engine Service

Overview

The main background application that runs continuously.

Responsibility

- Wait for market open
- Start trading session
- Stop trading session
- Repeat every trading day

Input

- Time
- Market events

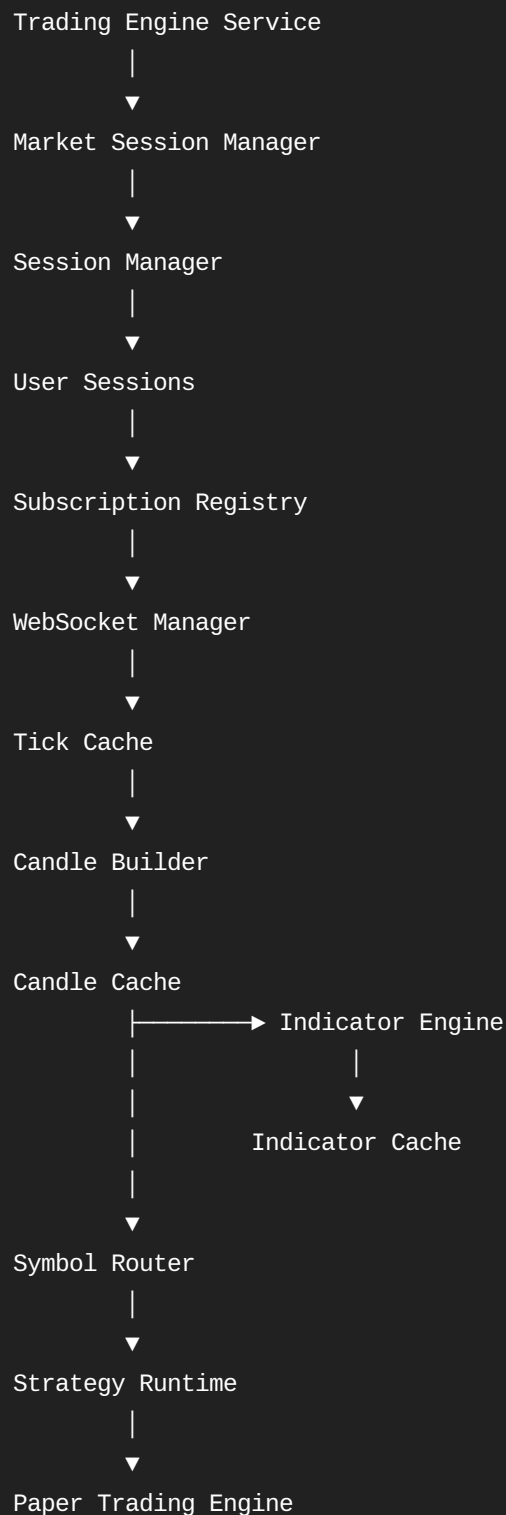
Output

- Starts and stops the complete trading system

Problem it solves

Keeps the trading engine running automatically without manual intervention.

Final Flow (Version 1)



One final recommendation before coding

Before writing **any** code for a module, let's create a one-page design document with these sections:

- **Purpose**
- **Responsibilities**
- **Inputs**
- **Outputs**
- **Public Methods**

- **Internal Data**
- **Dependencies**
- **Possible Failure Cases**
- **Unit Tests**

If we follow this process for each of the 14 modules, you'll not only end up with a working trading engine but also with documentation and design artifacts that resemble the engineering process used in many software companies. That will make the project much stronger as a portfolio piece for backend internships.